DATA ENGINEERING REPORT · PROJECT 3

Should we sell in Brazil?

A local modern data stack on 100,000 real orders:
S3 data lake, dbt, DuckDB, Dagster, Docker and continuous integration

LUMU BAMANA MANASSE

Data engineer

Date	October 2026
Version	1.0.0 (delivered and accepted)
Client	Maison Caju (fictional company) · Engagement CAJ-2026-001
GitHub repository	https://github.com/MediJr/olist-ecommerce-modern-data-stack
Language	English · *version française disponible dans le dépôt*

Fictional scenario, real data: the client company and its contacts are invented to simulate a complete professional engagement. The data is the *Brazilian E-Commerce Public Dataset by Olist*, published on Kaggle under the CC BY-NC-SA 4.0 licence (anonymised orders from 2016 to 2018). This project is not commissioned, endorsed or affiliated with Olist.

Document information

Project	Portfolio project 3: modern data stack (data lake, dbt, orchestration, CI)
Engagement	CAJ-2026-001: "Should we launch on Brazilian marketplaces?"
Client	Maison Caju, French home-textile brand (fictional), attention of Claire Dumont
Author	Lumu Bamana Manasse, data engineer
Period	October 2026
Data	Olist, 99,441 orders placed from September 2016 to October 2018
Status	DELIVERED ACCEPTED
Source code	https://github.com/MediJr/olist-ecommerce-modern-data-stack

Version history

Version	Date	Purpose
0.1	2026-10-23	Scoping: brief, meeting minutes, statement of work
0.5	2026-10-23	S3 lake, ingestion, dbt project, Dagster orchestration
0.9	2026-10-25	Whole platform moved to Docker, continuous integration, dashboard
1.0.0	2026-10-25	Acceptance on a fresh machine, final report (French and English)

Reading guide

Board	1. Executive summary and 11. Results and recommendations
Analyst (Hugo)	6. dbt data model, 7. Quality and tests, 9. Operations
Engineer taking over	4. Architecture, 5. Pipeline and orchestration, 13. Lessons learned, appendices

Contents

Document information 2

1 Executive summary 4

2 Context and scoping 5

2.1 What the request implies technically . . . 5
2.2 Scoping decisions . . . 5
2.3 Business questions . . . 5

3 Sources and profiling **6**
3.1 The dataset . . . 6
3.2 What profiling revealed . . . 6

4 Architecture **6**

5 Pipeline and orchestration **7**
5.1 The delivery contract . . . 7
5.2 The bronze layer: idempotent by design . . . 7
5.3 Event-driven orchestration . . . 8

6 dbt data model **8**
6.1 Three layers, three roles . . . 8
6.2 The star schema . . . 8
6.3 The rule that prevents multiplying revenue . . . 8

7 Quality and tests **9**
7.1 Continuous integration that bites . . . 9

8 Security and personal data **10**

9 Operations **10**

10 Acceptance **10**

11 Results and recommendations **11**

12 Limitations **15**

13 Lessons learned **16**

14 Roadmap **16**

A Repository structure **17**

B Continuous integration steps **17**

1 Executive summary

The need. Maison Caju is considering selling its collections in Brazil through local marketplaces. Its board wants a quantified market study, and its analyst must be able to **evolve the metrics alone, in SQL, without risk of regression**. Data arrives as **monthly deliveries**.

The solution. A complete **modern data stack** that fits on a laptop and starts with a single command, `docker compose up`. Each delivery is checked, then stored in an **S3 data lake** (Parquet, one partition per month). It is transformed by **dbt** on **DuckDB** into a tested star schema, orchestrated by **Dagster** (sensors, partitions, replays), and served to a bilingual dashboard. **Continuous integration** rebuilds and tests the whole chain on every change.

99,441 real orders (2016-2018)	**27 / 27** successful runs on a fresh machine	**18 min** from zero to dashboard	**94 / 94** dbt models and tests
0 duplicate on replay	**= source** revenue reconciled to the cent	**14** Python tests	**6** documented decisions (ADR)

Key results for Maison Caju

- **Q1 · Where to sell:** BRL 13.5 M in revenue. Home categories weigh **19.8%**, and bed & bath (*bed_bath_table*) is the 3rd category. São Paulo, Rio and Minas Gerais make **63%** of revenue.
- **Q2 · Loyalty:** only **3.0%** of customers order twice. This market is won through acquisition.
- **Q3 · Delivery:** the rating drops from **4.3 to 2.1** from 4 days late. The carrier choice is strategic.
- **Q4 · Payment:** credit card 75.5%, *boleto* 19.9%. Baskets triple with 7 to 10 instalments.
- **Q5 · Freight:** 13.8% of item price in São Paulo, up to **28.5%** in the North.

What this project demonstrates in data engineering: S3 data lake and partitioned Parquet, delivery contract, idempotent ingestion, layered dbt modelling with data tests, unit tests, contract and source reconciliations, event-driven orchestration with Dagster, concurrency management for a single-writer warehouse, full containerisation and continuous integration.

2 Context and scoping

2.1 What the request implies technically

Client request	Technical consequence
"Hugo must evolve the metrics without breaking what works"	Business logic in **versioned, tested SQL** (dbt), mandatory **continuous integration**
"A very detailed dataset" (9 tables)	**Dimensional model**: explicit grain, star schema
"A dashboard for the board"	It reads **pre-computed, tested** tables, never raw files
Monthly exports from the partner	**Partitions** per delivery, duplicate-free **replays**, an **orchestrator**

Table 1: From request to technical requirements

2.2 Scoping decisions

#	Topic	Decision
D1	Updates	One export per month; simulated by splitting the public dataset by purchase month
D2	Revenue	Item price, excluding freight, orders neither cancelled nor unavailable. Defined **once**, in dbt
D3	Repeat customer	Customer identified by `customer_unique_id`; repeat = at least two valid orders
D4	Dashboard	Metabase, later replaced by Streamlit (ADR 004)
D5	Maintenance	Every change goes through a pull request tested by continuous integration
D6	Language	Tables and columns in English; documentation in French, report also in English

Table 2: Decisions of the 2026-10-23 scoping meeting

2.3 Business questions

#	Question	Decision supported
Q1	Which categories and states drive revenue, and how does it evolve?	Launch collections and regions
Q2	What share of customers comes back?	Loyalty or acquisition
Q3	Does late delivery hurt the rating?	Carrier choice
Q4	How do customers pay?	Payment offer
Q5	How much does freight weigh by state?	Logistics cost by region

Table 3: The five questions

3 Sources and profiling

3.1 The dataset

The *Brazilian E-Commerce Public Dataset by Olist* describes about 100,000 anonymised orders placed on a Brazilian marketplace between 2016 and 2018, in **9 linked CSV files**: orders, items, payments, reviews, customers, sellers, products, geolocation and category translations. The client downloaded it from Kaggle (personal account required) into `data/source/`.

3.2 What profiling revealed

Finding	Risk if ignored, and treatment
`customer_id`: 99,441 values for 96,096 people (`customer_unique_id`)	It changes with every order: loyalty would look **zero**. Count on `customer_unique_id`
2,961 orders paid in several payments (up to 29); 9,803 orders with several items	An items × payments join **multiplies revenue**. Aggregate each table before joining
98,410 `review_id` for 99,224 reviews	`review_id` alone is not a key: composite key (`review_id`, `order_id`)
2 untranslated categories, 610 products without category	Empty labels in the dashboard: explicit, tested fallback (`unknown`)
775 orders without items; 8 "delivered" orders without delivery date	Explicitly excluded from revenue or lead times
1,000,163 GPS points for 19,015 zip prefixes, some outside Brazil	Median per prefix, after geographic filtering
November 2016: no orders; September-October 2018: 20 orders, almost all cancelled	An empty delivery is still a delivery; analyses on complete months (01/2017-08/2018)

Table 4: Profiling: seven traps, seven treatments

4 Architecture

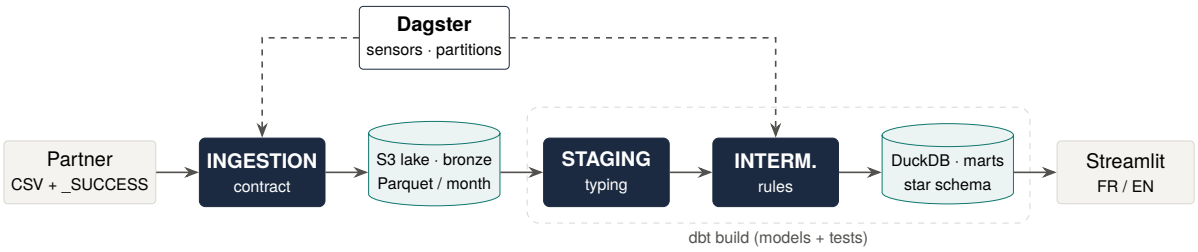


Figure 1: From delivery to dashboard

Role	Choice	Why
Data lake	S3-compatible storage (SeaweedFS), Parquet	Same interface as Amazon S3 or Google Cloud Storage: only the location changes to move to the cloud
Warehouse	DuckDB	Columnar, reads Parquet on S3 directly, nothing to administer
Transformations	dbt 1.12	Versioned, tested, documented SQL with lineage: maintainable by an analyst
Orchestration	Dagster 1.13	Thinks in *assets*; each dbt model becomes an asset and each test a check
Serving	Streamlit	Versioned, tested dashboard, read-only on the warehouse
Runtime	Docker Compose	Three services; single `python:3.13-slim` image, pinned versions
Quality	GitHub Actions	The whole chain rebuilt and tested on every pull request

Table 5: Technical choices (ADR 001, 002, 004, 005)

The lake is set by a single variable, `LAKE_URI`: `s3://olist-lake` in operation, a plain folder in continuous integration. **The code cannot tell the difference**, which is what makes it portable to the cloud.

5 Pipeline and orchestration

5.1 The delivery contract

The partner drops one folder per month and writes the `_SUCCESS` file **last**: a folder without this marker is a delivery in progress, never read. Before anything is written to the lake, each file must meet its contract:

- an **identical header**: an added, renamed or moved column blocks ingestion (schema drift);
- **non-empty, unique keys**;
- an **all-text** read with declared columns: an empty month (header only) reads like a full one.

5.2 The bronze layer: idempotent by design

Each table of a month becomes **a single Parquet file**, laid out with the Hive convention that DuckDB, Spark and BigQuery all read:

```
s3://olist-lake/bronze/orders/delivery_month=2017-01/orders.parquet
```

Replaying a month **replaces** that file: a duplicate is impossible. Two lineage columns are added: `_source_file` (original file) and `_ingested_at` (time it entered the lake).

5.3 Event-driven orchestration

Dagster mechanism	Role
Dynamic partitions	A `delivery_month` partition is created when a delivery is **received**, rather than a calendar fixed in advance
`landing_sensor`	Every 30 s: a new **or re-delivered** month (modified marker) triggers the `monthly_delivery` job
Run key `month@timestamp`	A given version of a delivery is never processed twice
Concurrency limit	DuckDB accepts a single writer: **one run at a time** (`dagster.yaml`)
Bootstrap	The monthly sensor waits for reference data; reference data arriving alone is loaded without dbt
Maximum runtime	15 minutes per run, monitored by the daemon: a hang no longer blocks the queue

Table 6: Orchestration (ADR 006)

6 dbt data model

6.1 Three layers, three roles

Layer	Models	Role
Staging (views)	9	One model per source: rename, type (`decimal` for money, never `float`), keep the latest delivery. **No business rule**
Intermediate (views)	2	`int_orders`: one row per order and **all** business rules, written once
Marts (tables)	6 + 5	Star schema, plus one reporting table per question

Table 7: The 22 dbt models (ADR 003)

6.2 The star schema

The main fact table, `fct_order_items`, has **the item sold as its grain**: the finest level that carries revenue. One can always aggregate up (order, category, month), never drill down. It is surrounded by `dim_customers` (real customer), `dim_products`, `dim_sellers` and `dim_date`. A second fact table, `fct_orders`, carries order-level measures: delivery, review, payment.

6.3 The rule that prevents multiplying revenue

An order with 2 items and 2 payments yields **4 rows** if raw tables are joined. `int_orders` therefore aggregates each child table **before** joining:

```
with items as (
    select order_id, count(*) as item_count, sum(price) as items_value,
```

```
            sum(freight_value) as freight_value
    from {{ ref('stg_order_items') }} group by order_id
),
payments as (
    select order_id, sum(payment_value) as payment_value, count(*) as payment_count
    from {{ ref('stg_order_payments') }} group by order_id
)
select ... from {{ ref('stg_orders') }} o
left join items using (order_id) left join payments using (order_id)
```

This rule is locked by a **dbt unit test** (two items, two payments, expected amounts) and by a **reconciliation** with the source (section 7).

7 Quality and tests

Type	What is checked	Count
dbt data tests	Uniqueness, not null, relationships, accepted values (statuses, ratings, 27 states)	69
dbt unit tests	Validity and delay; no amount multiplication; real-customer order rank	3
Reconciliations (singular tests)	Mart revenue = bronze total; payments = bronze total; full cohort at month 0	3
Enforced contract	Columns and types of `fct_order_items`	1
Python tests (pytest)	Delivery splitting, contract, idempotency, empty month, dashboard	14

Table 8: The safety nets

> **Reconciliation with the source**
>
> The `assert_revenue_reconciles_with_source` test recomputes revenue **directly on the bronze layer**, without any dbt model, and compares it to the dashboard's. Result: **BRL 13,494,400.74 on both sides**, to the cent. If a join loses or duplicates an item, the totals diverge and the build fails.

7.1 Continuous integration that bites

For acceptance, a classic mistake was introduced on purpose: **adding freight to revenue**.

dbt build · deliberate bug

```
# 1. freight added: the column type changes
Compilation Error in model fct_order_items
  This model has an enforced contract that failed.

# 2. freight added, keeping the same type
94 of 94 FAIL 1 assert_revenue_reconciles_with_source ......... [FAIL 1]
Done. PASS=93 WARN=0 ERROR=1 SKIP=0 TOTAL=94          (exit code: 1)
```

The two nets complement each other: the **contract** protects the shape (columns, types), the **reconciliation** protects the substance (amounts). In both cases dbt exits with an error, so the pull request would be rejected.

8 Security and personal data

- Data **anonymised by the publisher**: hashed identifiers, no names or addresses; only city, state and zip prefix remain.
- Secrets (S3 keys) in `.env`, excluded from Git; the S3 identity file is generated when the container starts.
- Services exposed on `127.0.0.1` only; containers run as an **unprivileged user**.
- The dashboard mounts the warehouse **read-only**.
- CC BY-NC-SA 4.0 licence: the full archive is not redistributed; only an attributed 450-order sample is used by CI.

9 Operations

Need	Command or tool
Start the platform	`docker compose up -d --build`
Receive a delivery	Drop the month's folder into `data/landing/`: the sensor does the rest
Replay a corrected month	Re-deliver the folder (new `_SUCCESS`) or re-run the partition in Dagster
Monitor runs	Dagster UI, http://127.0.0.1:3000: graph, partitions, checks, logs
Change a metric	Edit the dbt model, then `docker compose run --rm dagster python -m olist_pipeline.dbt_cli build`
Run Python tests	`docker compose run --rm dagster pytest`

Table 9: Day-to-day operations

10 Acceptance

Acceptance ran on a **simulated fresh machine**: Docker volumes deleted, empty landing zone. The 26 deliveries and the reference data were dropped at once, then processed with no inter-

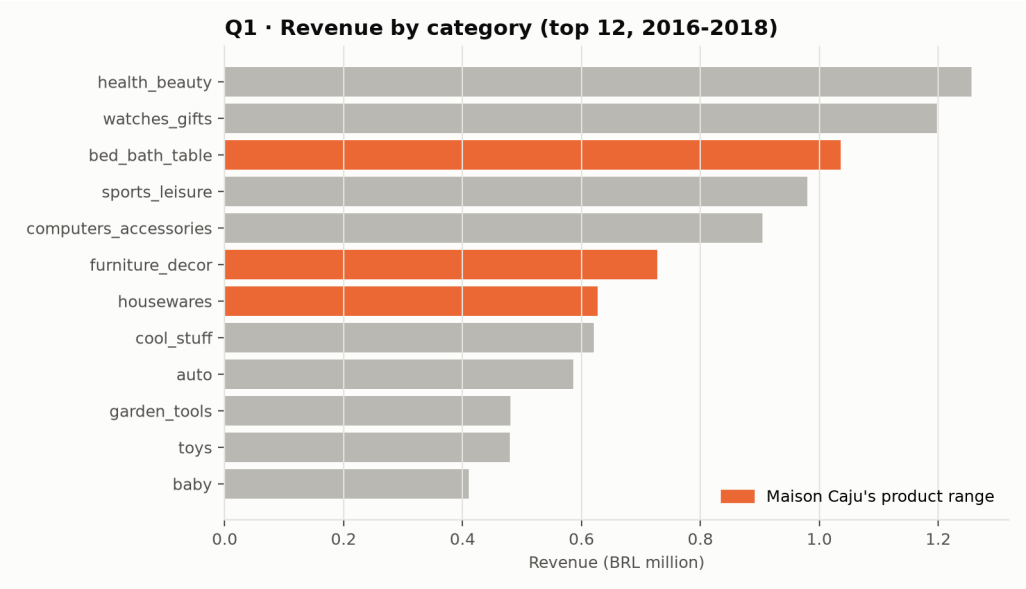
vention: **27 successful runs out of 27, in 18 minutes**, with a median of 19 s of processing per month.

#	Criterion	Result
C1	Starts on a fresh machine by following the README	✓
C2	One month = one partition; processing a month does not reprocess the others	✓
C3	Replaying a month creates no duplicate (October 2016 re-delivered: 329 orders before and after)	✓
C4	Mart revenue equals the source total (BRL 13,494,400.74)	✓
C5	All dbt tests pass (94/94); every model is documented	✓
C6	CI fails when a test breaks (demonstration above)	✓
C7	The dashboard answers Q1-Q5 (5 tabs, 10 charts, FR/EN)	✓
C8	No secret or raw data in version control	✓

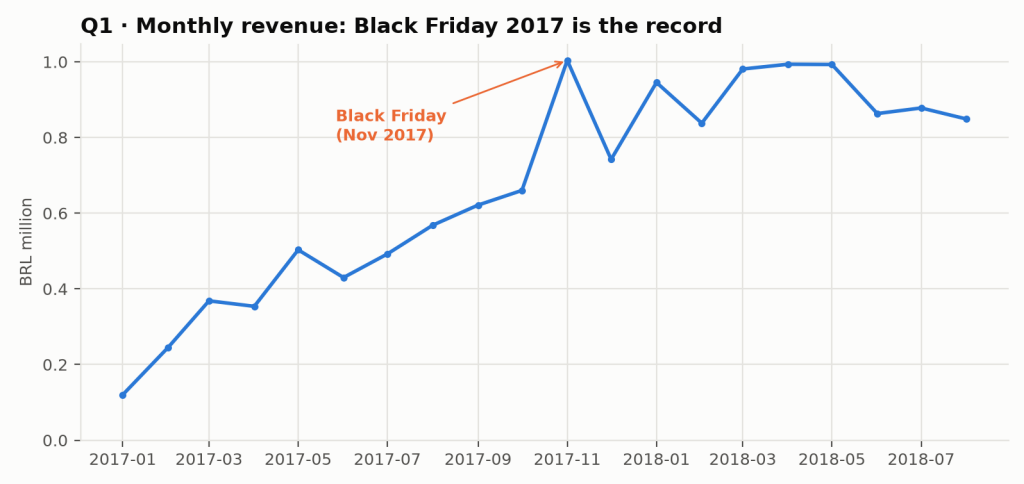
Table 10: Acceptance criteria: 8 out of 8

11 Results and recommendations

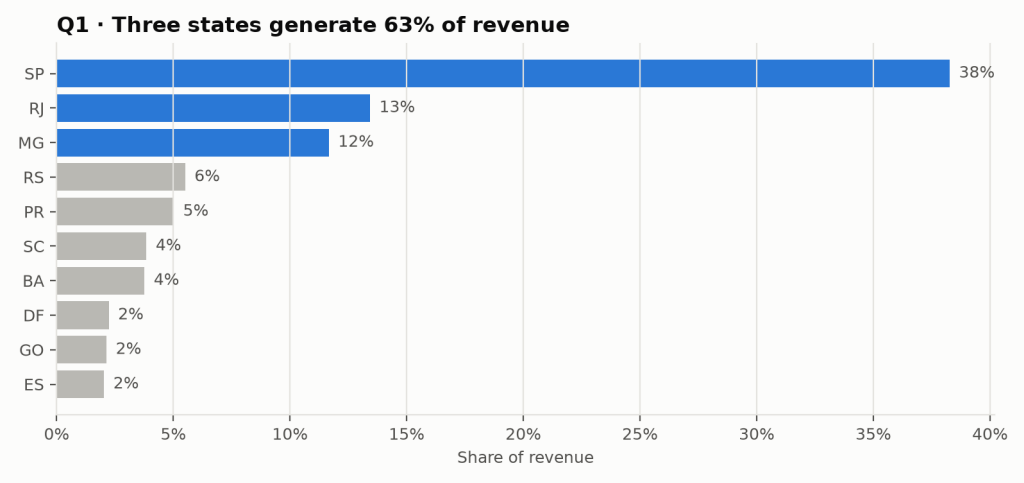
Q1 · Where is revenue made?



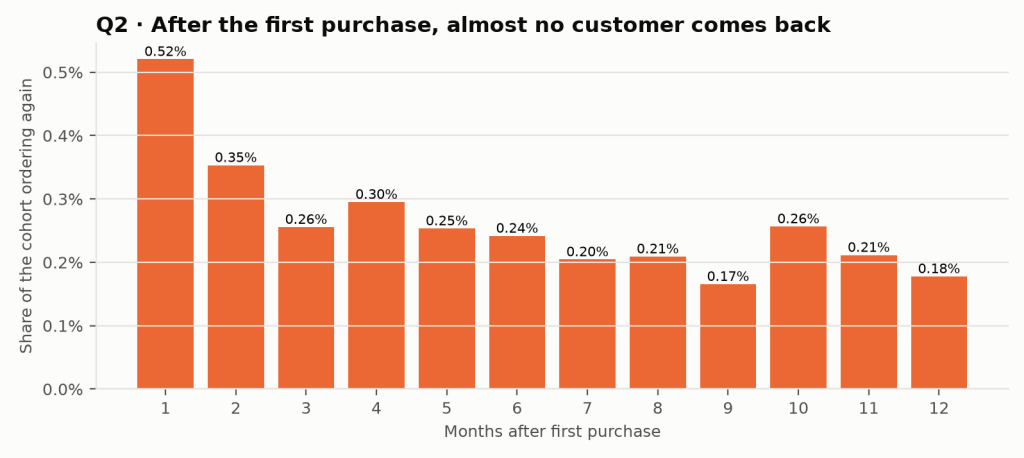
Across 98,207 valid orders, revenue reaches **BRL 13.5 M** (excluding freight). Maison Caju's product range weighs **19.8%**, and bed & bath (*bed_bath_table*, 7.7%) is the market's 3rd category.



The market grew strongly in 2017, then stabilised around BRL 0.9 M per month in 2018. **Black Friday 2017** beat October by **52%**.

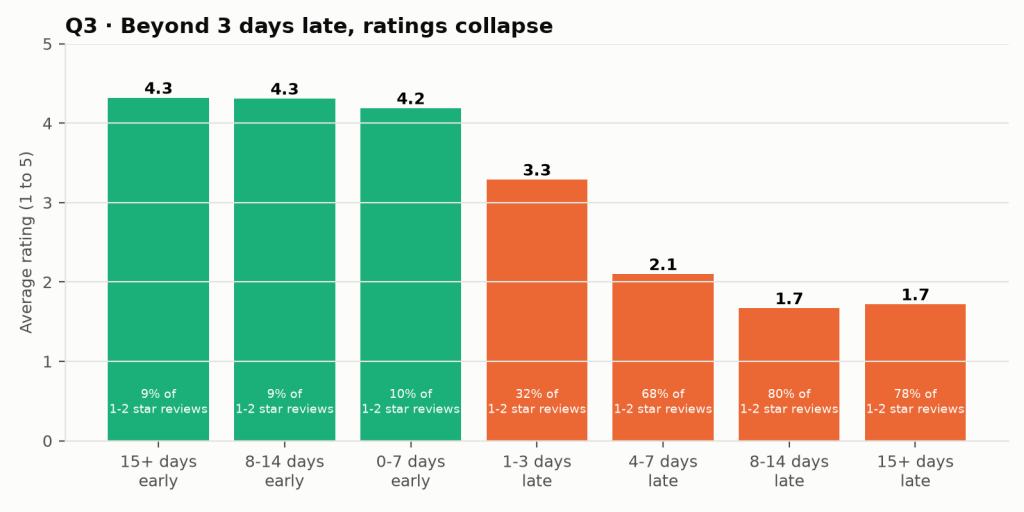


Q2 · Do customers come back?



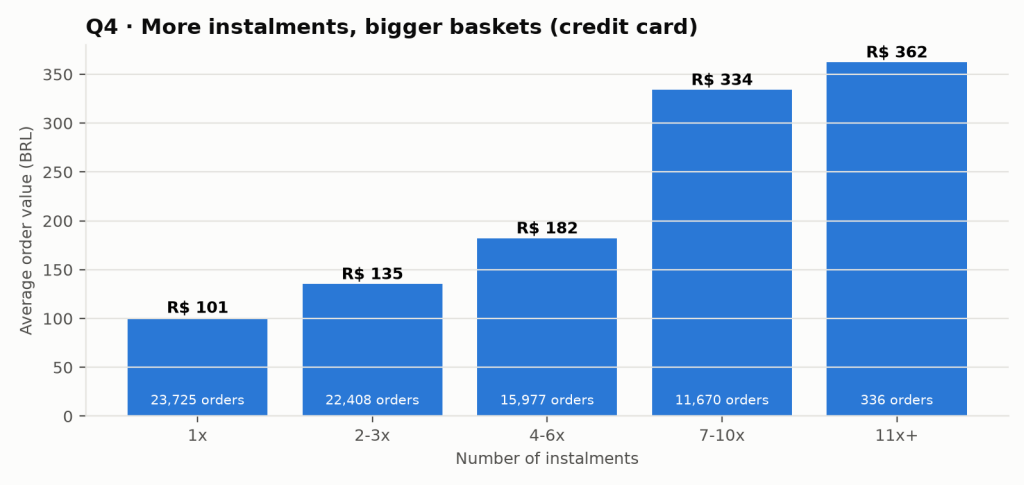
Of 94,990 customers with a valid order, only **2,888** (**3.0%**) placed a second one, a median of 28 days later. One month after their first purchase, barely **0.5%** of 2017 customers order again.

Q3 · Does late delivery cost reviews?



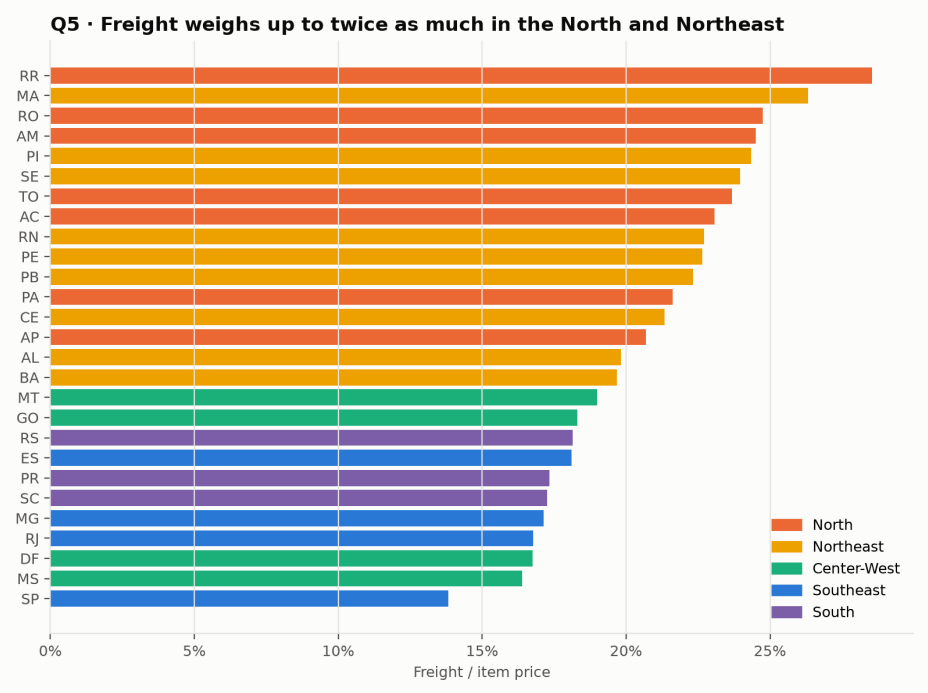
6.8% of delivered orders arrive late. The average rating falls from **4.3** (on time or early) to **2.1** at 4 to 7 days late. Beyond a week, **80%** of reviews are 1 or 2 stars.

Q4 · How do customers pay?



Credit cards settle 75.5% of orders, *boleto* (a payment slip paid in cash or online) 19.9%. With a card, the average basket rises from **BRL 101** in one instalment to **BRL 334** in 7 to 10 instalments.

Q5 · How much does freight weigh?



Freight is 13.8% of item price in São Paulo (8.7 days average delivery), but **up to 28.5%** in Roraima (29 days). The North and Northeast combine high freight, long lead times and more frequent delays.

Recommendations to the board

1. **Launch with bed & bath and home decor**, already the market's 3rd and 6th categories.
2. **Start with the Southeast** (São Paulo, Rio, Minas Gerais): 63% of revenue, the lowest freight and the shortest lead times. The North and Northeast come next, with a dedicated freight policy.
3. **Invest in acquisition rather than loyalty**: 97% of customers buy only once. Prepare for Black Friday.
4. **Choose the carrier on punctuality** and promise realistic lead times: a few days late halves the rating.
5. **Offer instalments and *boleto***: the average basket triples with instalments.

12 Limitations

- The data covers 2016-2018: it describes the market's **structure**, not its current state, and comes from a single marketplace.
- Revenue is that of all the marketplace's sellers, not of a brand comparable to Maison Caju.
- The warehouse is rebuilt at each delivery (about 20 s): right for this volume, not for hundreds of millions of rows.
- `dagster dev` runs server and daemon in one container with SQLite history: right for a laptop, not for production.

13 Lessons learned

Incident	Lesson
MinIO image no longer published	An open-source component can change its distribution model. Thanks to the S3 standard, SeaweedFS replaced it **without changing a line of code** (ADR 002)
dbt run before the first order	A platform's start-up order is a dependency like any other: it must be **explicit** (bootstrap, ADR 006)
`dagster-dbt` requires Python < 3.14	Check the compatibility of the **whole** chain before choosing a version
Smart App Control blocks unsigned DLLs; a run hung for 83 min and blocked the queue	A single-run queue needs a **maximum runtime**. Containerisation makes the platform independent of the machine (ADR 005)
Dashboard test fails in CI	The default period was hard-coded: a test on different data reveals **hidden assumptions**
Delivery simulation took 3 min 30	Filtering a table 26 times costs more than a single group-by: 26 s after the fix

Table 11: What this project taught

14 Roadmap

1. **Move to the cloud** (project 6): lake on Google Cloud Storage, BigQuery warehouse, same dbt structure, infrastructure as code.
2. **Incremental dbt models** if volume or frequency grows.
3. **Production Dagster**: separate server, daemon and PostgreSQL storage; failure alerts.
4. **Source freshness**: alert if no delivery arrives within 35 days (`dbt source freshness`, already configured).
5. **Review analysis** (Portuguese text) to understand the causes of dissatisfaction.

A Repository structure

```
src/olist_pipeline/   config, landing, bronze, definitions (Dagster), dbt_cli, report
dbt/models/           staging (9) · intermediate (2) · marts/core (6) · marts/reporting (5)
dbt/tests/            3 reconciliations with the source
dashboard/app.py      Streamlit dashboard (FR/EN)
tests/                pytest + CI sample (450 real orders)
dagster_home/         Dagster configuration (concurrency, run monitoring)
docs/                 engagement, ADRs, work log, data dictionary, reports, acceptance
Dockerfile · docker-compose.yml · .github/workflows/ci.yml
```

B Continuous integration steps

1. Code style (`ruff`) and Python tests (`pytest`).
2. Simulated deliveries from the sample, then ingestion into a local lake (folder).
3. `dbt build`: 22 models, 69 data tests, 3 unit tests.
4. Dagster definitions validation, dashboard smoke test.
5. Docker image build.